Computer Architecture 2024fall Project 3

> *By Prof. Jie Zhang*
>
> **Due:** 11:59:59 pm, December 17, 2024

In this project, we will explore the design of a cache prefetcher in a computer system. Specifically, we will first introduce how to build a cache hierarchy in the configuration script, and how to add a prefetcher to a specific cache. You will evaluate how caches and prefetcher can significantly impact system performance. Further, you are required to read a recent paper that proposes a prefetcher named Hyperion. You need to summarize the paper and implement the Hyperion in `gem5`. Next, you are required to compare your prefetcher's performance with that of other prefetchers in `gem5`.

Whenever you face trouble or problems with this project, please contact TAs by WeChat or e-mail. All the following information has been verified on Ubuntu 20.04.1 with Linux Kernel 5.15.0 and Ubuntu 22.04.2 LTS with Linux Kernel 5.19.0.

Part 1: Add Caches in the Configuration Script

In this part, we will extend our configuration script with a cache hierarchy. Let's start with copying the original script to a new directory:

```
mkdir -p configs/proj3
cp configs/proj1/simple.py configs/proj3
```

In this part, we will add `SimObject` of caches to the CPU. `gem5` provides a comprehensive cache implementation, found in `src/mem/cache/Cache.py` . An abstract class `BaseCache` describes the basic cache framework. There are seven critical parameters that define the performance of a given cache:

- `size` : the total size in bytes of the cache.
- `assoc` : the associativity of the cache, namely the number of blocks (cachelines) in a single cache set.
- `tag_latency` : the cycles to lookup the tag in the cache.
- `data_latency` : the cycles to access the data in the cache.
- `response_latency` : the cycles for the return path on a miss.
- `mshrs` : the number of MSHRs (Miss Status Holding Register).
- `tgts_per_mshr` : the max number of accesses per MSHR.

To create `SimObject` of caches, we need to define our specific non-abstract classes first. Let's add the following classes at the beginning of our configuration script (right after the `import` statements):

```
class L1D(Cache):
    size = '48kB'
    assoc = 12
    tag_latency = 3
    data_latency = 3
    response_latency = 3
    mshrs = 16
    tgts_per_mshr = 20

class L1I(L1D):
```

```
    pass

class L2(Cache):
    size = '2MB'
    assoc = 16
    tag_latency = 20
    data_latency = 20
    response_latency = 20
    mshrs = 20
    tgts_per_mshr = 12
```

Once we have the cache classes, we can build a "two-level" cache hierarchy in our script. In this part, we will use the helpers provided by `gem5` to make our script simple. Specifically, our function `init_system()` can be modified like follows:

```
def init_system(system, args):
    system.clk_domain = SrcClockDomain(clock='4GHz',
        voltage_domain=VoltageDomain())
    system.mem_mode = 'timing'
    system.mem_ranges = [AddrRange ('2GB')]
    system.mem_ctrl = MemCtrl()
    system.mem_ctrl.dram = DDR4_2400_8x8()
    system.mem_ctrl.dram.range = root.system.mem_ranges[0]
    system.membus = SystemXBar()
    # Use O3CPU as default CPU, similar to project 2
    system.cpu = O3CPU()
    system.cpu.createInterruptController()
    # MODIFIED: use addTwoLevelCacheHierarchy() and connectBus()
    system.cpu.addTwoLevelCacheHierarchy(L1I(), L1D(), L2())
    system.cpu.connectBus(system.membus)
    system.mem_ctrl.port = system.membus.mem_side_ports
    system.system_port = system.membus.cpu_side_ports
```

By using `addTwoLevelCacheHierarchy()` and `connectBus()`, we can add a cache hierarchy to the CPU without manually connecting all the ports and buses.

Now, if you start a simulation with your new script (for example, running `tests\labexe\hello`), you can find that the caches have been added to CPU in `config.ini`:

```
# config.ini
...
[system.cpu]
type=BaseO3CPU
```

```
children=...dcache icache l2cache...
...
```

Deliverables

Please run the memory-sensitive workload, i.e., `gemm`, with the two-level cache hierarchy we defined above. In this part, please include the following contents in your ***report***:

- A screenshot of `config.ini` that shows you have successfully added the cache hierarchy.
- A figure or table, comparing the `system.cpu.ipc` on workload `gemm` of system with and without caches. You can use the results in project 1 and 2 as the results of cache-free systems.

Part 2: Run Simulation with Prefetchers

`gem5` provides multiple classic cache prefetchers. They can be found in `src/mem/cache/prefetch/Prefetcher.py`. One of the simplest prefetchers is the `StridePrefetcher`. We can directly add it to the CPU's data cache with the following code:

```
def init_system(system, args):
    ...
    system.cpu.addTwoLevelCacheHierarchy(L1I(), L1D(), L2())
    # ADD FOLLOWS:
    system.cpu.dcache.prefetcher = StridePrefetcher()
    ...
```

When you start a simulation, you can find the prefetcher in `config.ini`:

```
# config.ini
...
[system.cpu.dcache.prefetcher]
type=StridePrefetcher
...
```

To evaluate different prefetchers, we can also add options in the script. In this part, we will explore two of the provided prefetchers: `StridePrefetcher` and `AMPMPrefetcher`:

```
pf_types = {
    "stride": StridePrefetcher,
    "ampm": AMPMPrefetcher,
}
...
def init_system(system, args):
    ...
    system.cpu.dcache.prefetcher = pf_types[args.pf]()
    ...
...
if __name__ == '__m5_main__':
    ...
    parser.add_argument(
        "--pf", type=str, default="stride",
    )
    ...
```

Then, let's run the test executables and learn how these prefetchers can impact on system performance. Among the provided executables, `shell_sort` and `gemm` are sensitive to prefetching. In project 3, you only need to run the sensitive workloads, that is, `shell_sort` and `gemm`. So, to test the performance of stride prefetcher, the commands are as follows:

```
# using default CPU and Mem
build/ARM/gem5.opt -d m5out/stride/sort configs/proj3/simple.py
tests/labexe/shell_sort --pf stride
build/ARM/gem5.opt -d m5out/stride/gemm configs/proj3/simple.py
tests/labexe/gemm --pf stride
```

The commands to test AMPM are similar.

> ⚠ **Important note**
>
> In previous project, the recommended `system.cpu.max_insts_any_thread` is 1e8. This value makes sure you can run a simulation fastly. However, typical prefetcher requires much more accesses to learn the pattern and generate prefetches. Thus, in this project, ***the recommended `system.cpu.max_insts_any_thread` is 1e9***. You can use 1e8 or even less values to speed-up your debugging. We also **accept** reports that are generated from less values, but the statistic outputs may be unreasonable. If you face troubles (for example, you cannot complete simulation with value 1e9), you can inform us in your report, the grading will be discretionary.

After the simulations have finished, let's check the statistics in `stats.txt`. In this project, we care about four metrics, which evaluate the performance of the prefetchers:

- `cpu.ipc` : The IPC of the cpu.
- `prefetcher.accuracy` : The accuracy of the prefetcher, which means the fraction of useful prefetches among all prefetches (*check appendix A for detailed explaination*).
- `prefetcher.coverage` : The coverage of the prefetcher, which means the fraction of eliminated misses (first-time hits on prefetched data) among all potential misses (eliminated misses plus actual misses).
- `prefetcher.pfLate` : The number of prefetches that are issued lately (data are already in cache, MSHRs or write buffer before issuing).

Delieverables

Please run two prefetch-sensitive workloads, i.e., `shell_sort` and `gemm`, with two prefetchers. The rest workloads `spfa` and `binary_search`, are insensitive to prefetching. You can run these workloads if you are interested, and have sufficient computing resources. Nevertheless, in this part, please (at least) include figures or tables, which show the four critical metrics (i.e., IPC, accuracy, coverage, late count) of different prefetch configurations on workloads listed below, in your ***report***:

- prefetch: **No prefetcher**, `StridePrefetcher`, `AMPMPrefetcher`.
- exe: `shell_sort`, `gemm`.
- With default CPU and Memory (O3 and DDR4).
- Use the cache hierarchy described in part 1, and attach the prefetchers to `cpu.dcache`.

In total, you need to run `gem5` six times.

Part 3: Read Paper, Implement and Evaluate Hyperion Prefetcher

Implemeting a prefetcher in gem5

Compared with implementing branch predictors, as we did in project 2, implementing a prefetcher in `gem5` involves far fewer interfaces. Let's learn from the code of the simplest prefetcher in `gem5`, namely the `TaggedPrefetcher`.

```
// src/mem/cache/prefetch/tagged.hh
namespace gem5 {
namespace prefetch {
class Tagged : public Queued {
  protected:
      const int degree;

  public:
    Tagged(const TaggedPrefetcherParams &p);
    ~Tagged() = default;

    void calculatePrefetch(const PrefetchInfo &pfi,
                           std::vector<AddrPriority> &addresses,
                           const CacheAccessor &cache) override;
};
} // namespace prefetch
} // namespace gem5
```

The class `Tagged` is inherited from `prefetch::Queued`, which is inherited from `prefetch::Base`. `Queued` provides a framework that handles basic issues, like merging prefetches targeting the same address. Thus, our implementation can simply focus on generating prefetches, which is performed in the interface `calculatePrefetch()`.

If we check the code of `Base` (in `mem/cache/prefetch/base.hh`), it leaves multiple virtual methods, two of them are critical to our implementation:

- `notify()` : This is called when the cache is accessed. By default, two types of access will call this method, namely cache miss and hit on prefetched data (i.e., all potential misses). Set the parameter `prefetch_on_access = True` will call this method on each cache access.
- `notifyFill()` : This is called when a cacheline is filled into the cache. The cacheline may be demand missed or prefetched previously.

In the implementation of `Queued` (in `queued.hh`), the basic works, like merging prefetches, are wrapped in the default implementation of `notify()`. Instead, it leaves another virtual method, `calculatePrefetch()`. Let's check the implementation of `Tagged` to understand the implementation principle of this method:

```
// src/mem/cache/prefetch/tagged.cc
void Tagged::calculatePrefetch(const PrefetchInfo &pfi,
    std::vector<AddrPriority> &addresses, const CacheAccessor &cache) {
    Addr blkAddr = blockAddress(pfi.getAddr());

    for (int d = 1; d <= degree; d++) {
        Addr newAddr = blkAddr + d*(blkSize);
        addresses.push_back(AddrPriority(newAddr,0));
    }
}
```

Basically, the implementation will extract information from `pfi` and `cache`, calculate the addresses that are going to be prefetched, and push them into `addresses` using `push_back()`.

The defination of `PrefetchInfo` can be found in `base.hh`. Four interfaces are useful for our implementation:

- `getAddr()` : Get the address of the access that trigger the prefetcher.
- `hasPC()` : Check if this access is performed by an instruction.
- `getPC()` : Get the PC of the instruction that performs this access.
- `isCacheMiss()` : Check if this access is a cache miss.

The `Base` also provides several useful tools for extract the information:

- `blockAddress()` : Extract the block base of an address. For example, if block size is 64 Bytes, and address `A = 66`, then `blockAddress(A) = 64`.

- `blockIndex()` : Extract the block index of an address. For example, `blockIndex(A) = 1` .
- `pageAddress()` : Extract the page base of an address.
- `pageOffset()` : Extract the offset of an address in its page.

Please note that `Base` doesn't provide a method like `pageIndex()` . By default, the page size is configured to `4kiB` . Thus, you can implement this method on your own (for example, `pageIndex(Addr a) { return a >> 12; }`).

The `CacheAccessor` provides some interfaces to allow you check the cache status (for example, whether an address is in the cache), they are defined in `src/mem/cache/cache_probe_arg.hh` .

`Tagged` doesn't uses `notifyFill()` . We can found the defination in `base.hh` :

```
// src/mem/cache/prefetch/base.hh
virtual void notifyFill(const CacheAccessProbeArg &acc) {}
```

The `CacheAccessProbeArg` records the memory packet that triggers the fill event, and a `CacheAccessor` that allows you to check the cache. This class can also be found in `src/mem/cache/cache_probe_arg.hh` . For the packet, `CacheAccessProbeArg` provide a pointer `PacketPtr pkt` . The two useful interfaces are:

- `pkt->getAddr()` : get the address of this data fill.
- `pkt->cmd.isHWPrefetech()` : check if the filled data are prefetched from your hardware prefetcher.

Please note that, when `notifyFill()` is called, the address of the filled data is **the block base address**. Thus, when you want to record accesses and check them in `notifyFill()` , you may want to record them in their block base address.

> **Info**
>
> If you check the defination of `CacheAccessor` and `PrefetchInfo` , you may find that there are more information recorded, including `isSecure` and `requestorID` . In this project, you can simply ignore them. You may only use the interfaces that doesn't require specifying `requestorID` , and always treat `isSecure` as `false` .

> For example, to check whether an address has been prefetched, you can just use `cache.hasBeenPrefetched(addr, false)` .
>
> Also, in next part, you will need to record the "timestamp" of several events. In `gem5` , we can disrectly use the function `curTick()` to get the exact current timestamp. An `uint64_t` value will be returned.

In summary, you only need to work with `calculatePrefetch()` and `notifyFull()` when you are implementing prefetcher in this part. The provided arguments can help you to determine what type of access (hit on prefetch, data fill or demand miss) calls these methods. All the structures can be implemented as private members and methods, ensuring high flexibility.

Reading the paper and implementing Hyperion prefetcher

> **Cite**
>
> Cui, Y., Chen, W., Cheng, X., & Yi, J. (2024). Hyperion: A Highly Effective Page and PC Based Delta Prefetcher. ACM Transactions on Architecture and Code Optimization.
>
> get the pdf

In this part, you are required to read the paper cited above, summarize the method described in the paper, and implement the Hyperion prefetcher in `gem5` . For your convenience, an Appendix provides a brief description of the method. We've already provided a code skeleton. Please check `Lab3Hyperion` in `Prefetcher.py` , `mem/cache/prefetch/lab3_pf.hh` and `mem/cache/prefetch/lab3_pf.cc` . You can follow the tutorial in project 2 to add useful parameters to `Lab3Hyperion` and implement C++ codes.

> **Delieverables**

> Please run two prefetch-sensitive workloads, i.e., `shell_sort` and `gemm`, with your Hyperion prefetcher. Please (at least) include figures or tables, which show the IPC of CPU, accuracy, coverage and late count of `Lab3Hyperion` on executables listed below, in your ***report***:
>
> - prefetch: `Lab3Hyperion`.
> - exe: `shell_sort`, `gemm`.
> - With default CPU and Memory (O3 and DDR4).
> - Use the cache hierarchy described in part 1, and attach the prefetcher to `cpu.dcache`.
>
> In total you need to run `gem5` twice.
>
> In this part, you also need to submit:
>
> - ***your codes***, including `Pretecher.py`, `lab3_pf.hh` and `lab3_pf.cc`.
> - ***paper summary***, which summarizes the cited paper, including background and designs (the motivation is not required considering the paper length).

Submission

Submission

- Please submit the deliverables for each part in order and clearly defined in a report form (e.g., PDF or Word). Please give a brief description of the results in your report. You also **NEED** to submit your codes and paper summary.
- Please pack your report, codes and summary in an archive and submit it as an attachment at course.pku.edu.cn. The title of the attachment should be Student-ID_Name_Proj3 (e.g., 123456789_WangXiaoming_Proj3).
- You can also send the archive to ca2024fall@163.com. The titles of your email AND attachment should both be Student-ID_Name_Proj3.
- **DO NOT PLAGIARIZE**. We will select 10 students randomly and ask them to answer our questions related to their results.

Late policy

- You will be given **3 slip days** (shared by all projects), which can be used to extend project deadlines, e.g., 1 project extended by 3 days or 3 projects each extended by 1 day.
- Projects are due at 23:59:59, no exceptions; 20% off per day late, 1 second late = 1 hour late = 1 day late.

Deliverables

You need to submit ***report***, ***codes*** and ***paper summary*** in this project. The required contents are listed as follows:

- report:
- - Figures and tables which show the CPU IPC and the accuracy, coverage and late count of following prefetch configurations on `shell_sort` and `gemm`: no prefetcher, `StridePrefetcher`, `AMPMPrefetcher`, `Lab3Hyperion`. Please run with `O3CPU` and `DDR4` memory, and with the cache configurations described in part 1.

- codes:
- - Your implementation, i.e., `Prefetcher.py`, `lab3_pf.hh` and `lab3_pf.cc`.
- summary:
- - A summary of the cited paper including background and designs.

Please pack all files in an archive and submit the archive.

✔ Grading

In this project, the grading is partially based on the performance of your implementation. The details are (15 points in total):

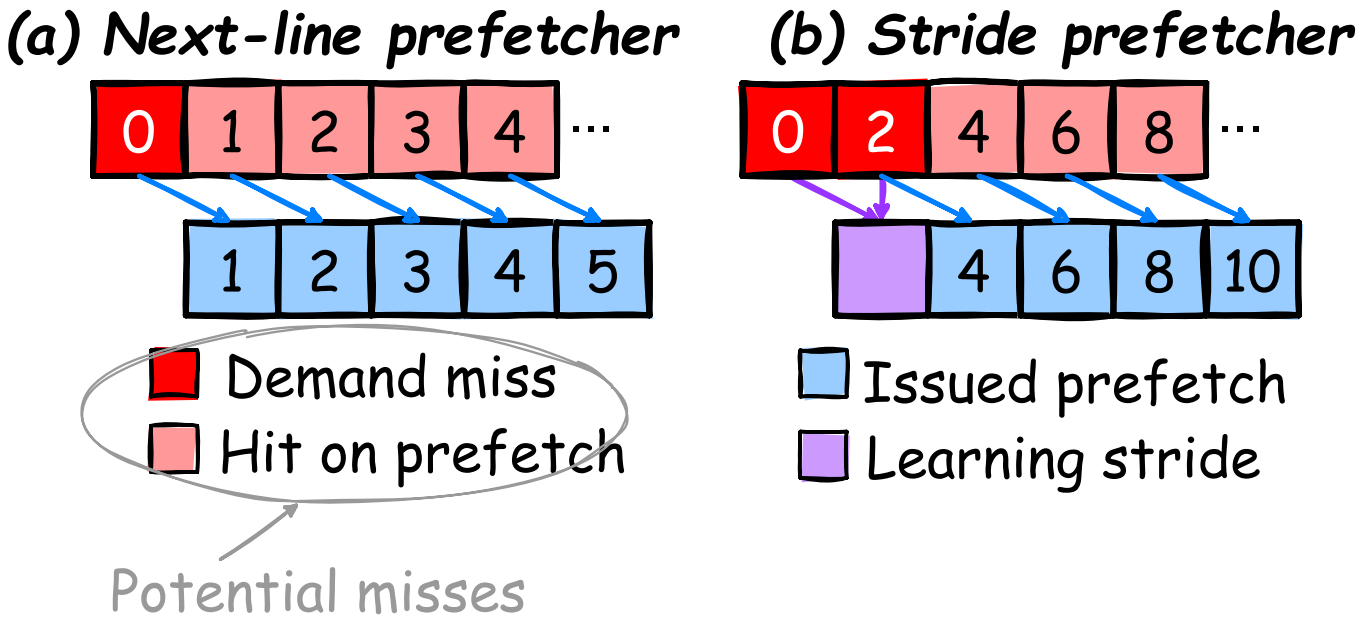
- Screenshot that proves you have successfully built a cache hierarchy:
- - **5pts**.
- ***PLUS*** resonable performance statistics of **no prefetcher**, `StridePrefetcher` and `AMPMPrefetcher`:
- - **9pts**.
- ***PLUS*** a compilable Hyperion prefetcher implementation:
- - **12pts**.
- ***PLUS*** a comparable performance with `StridePrefetcher` of your Hyperion prefetcher:
- - **14pts**.
- ***PLUS*** a comparable performance with `AMPMPrefetcher` of your Hyperion prefetcher:
- - **15pts**.

There are additional **5pts** based on your paper summary. A comprehensive summary including background and designs will get 5pts. Lack of any single part will cause **-1pt**.

Appendix A: Introduction to Cache Prefetchers

In modern memory hierarchy, the penalty of cache misses can be extremely large. This motivates designers to reduce the cache miss rate. Typically, some workloads show predictable memory access patterns, like sequentially accessing an array (address sequence as `A, A+1, ...`). In such cases, a previous access can inform the CPU about the potential following accesses (e.g., `A` can lead to `A+1`). The cache prefetchers are designed for these scenarios, which extract the workloads' access patterns, and load the following data before the CPU explicitly accesses them, eliminating cache misses.

The simplest implementation of prefetcher is the "next-line prefetcher", as shown in Fig (a) below. When the CPU access a block (cacheline) `A`, the next-line prefetcher will load the next block `A+1` to the cache. Note that in cache hierarchy, the access granualrity will always be block (typically 64B), so there is no concept like "next-byte prefetcher".



However, next-line prefetcher cannot perform well on some workloads. As a simple example, some workloads access the memory in a strided pattern, such as `A, A+2, A+4,...` (here we have `stride = 2`). In such scenarios, next-line prefetcher will never prefetch the being-used data. To serve these workloads, "stride prefetcher", as shown in Fig

(b), is proposed, which can *learn* the actual stride from the memory access history, and perform useful prefetches.

The "degree" denotes the depth of prefetching. For example, the next-line prefetcher has `degree = 1`, for it prefetches one line per issue. If it has larger degree, it can be named as "next-N-line prefetcher".

Metrics to evaluate prefetchers

In the text above, we claim that stride prefetcher can perform better than next-line prefetcher. As a question, how can we determine the "performance" of a prefetcher? Basically, we have several values or metrics to evaluate prefetchers. Here we list them with their name in `stats.txt` of `gem5` output:

- `pfIssued`: The number of prefetches issued by prefetcher.
- `pfUseful`: The number of prefetches that are useful (i.e., the prefetched data are accessed in the cache before being evicted).
- `pfUnused`: The number of prefetches that actually fill data into cache, but the data are not accessed before being evicted.
- `pfLate`: The number of prefetches that are issued lately (the prefetched data are in cache, MSHRs or write buffer).
- `demandMshrMisses`: The number of cache misses, no prefetch were issued to cover them.
- `accuracy`: The accuracy of the prefetcher, equivalent to `pfUseful / pfIssued`.
- `coverage`: The coverage of the prefetcher, equivalent to `pfUseful / potentialMisses`, where `potentialMisses = pfUseful + demandMshrMisses`. Here, "potential misses" refers to the accesses that would miss if there were no prefetcher. Therefore, the number of potential misses is the number of useful prefetches plus actual misses.

The most important metrics are `accuracy` and `coverage`. There is a trade-off between them. More aggressive prefetching will fetch more addresses, whether useful of not. Many of the actual accesses can be covered by prefetches, but many useless data are also prefetched, thus, increasing the `coverage` but decreasing `accuracy`. More conservative prefetching may increase `accuracy` but decrease `coverage`.

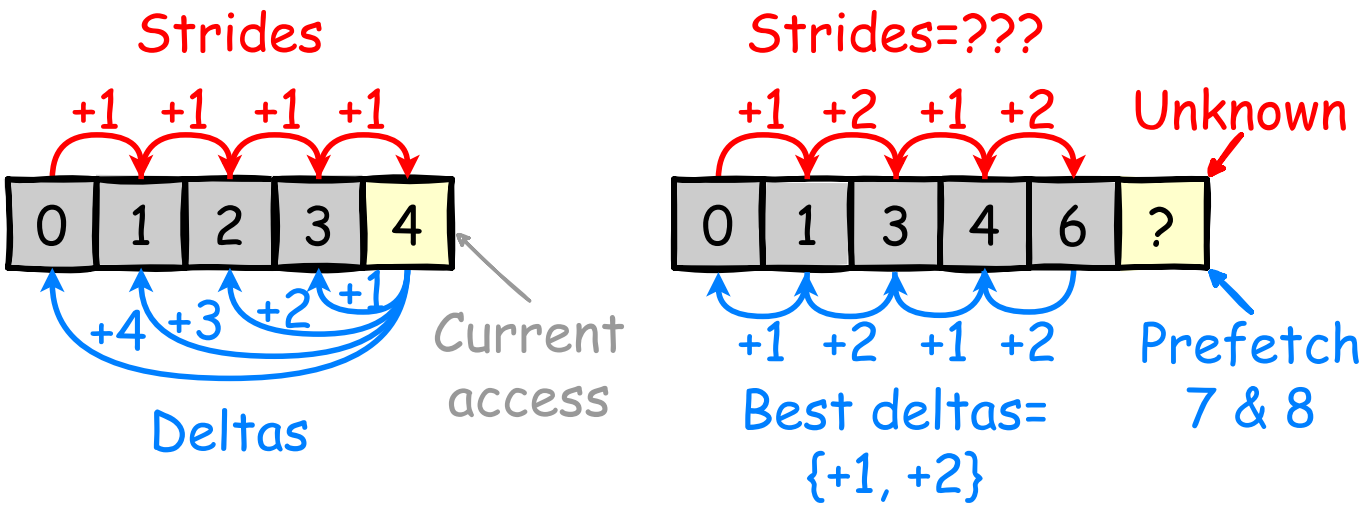
With these metrics, we can compare between next-line prefetcher and stride prefetcher: stride prefetcher can have higher `accuracy` and `coverage` than next-line prefetcher under strided accesses, since next-line prefetcher cannot prefetch any useful data.

Appendix B: Hyperion Prefetcher Brief Description

In this appendix, we will briefly describe the concepts of delta prefetching and timely prefetch. Based on these information, we describe the Hyperion prefetcher. Finally, we list the features that are proposed in the paper but you don't need to implement in your codes.

Delta prefetching and local-delta prefetching

To introduce delta prefetching, let's first consider a workload that performs an "irregular" memory access pattern: `A, A+1, A+3, A+4, A+6, ...`. The stride sequence of this stream is `+1, +2, +1, +2, ...`. Patterns that vary their strides like this are called irregular patterns. A normal stride prefetcher (*check Appendix A for details*) cannot provide any coverage for irregular patterns, since it cannot learn enough confidence for any individual stride value.



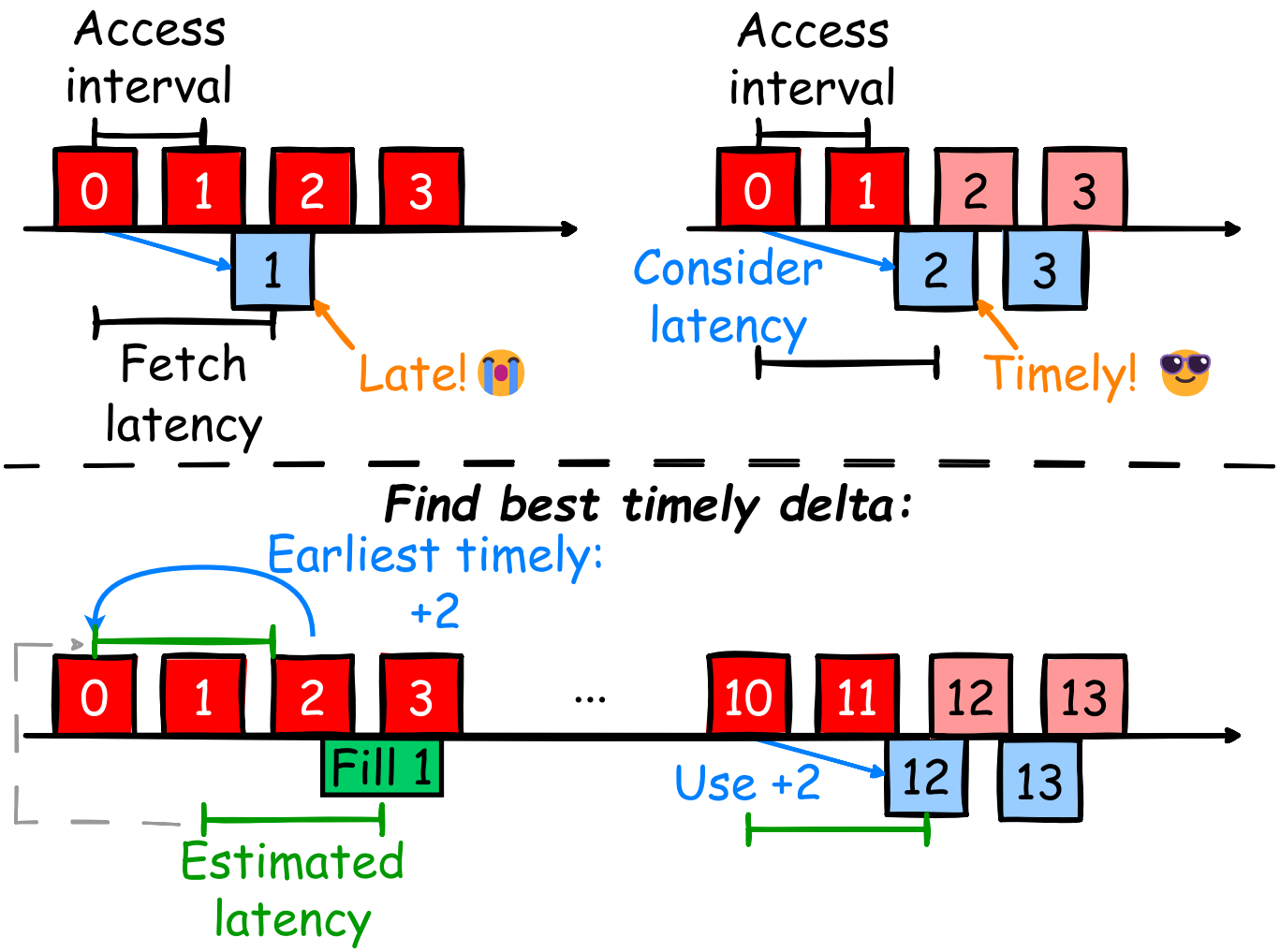
Delta prefetching is proposed to handle these irregular patterns. Conceptually, a "delta" is the address difference of current demand access with any previous accesses, as shown in the left figure above. The principle of delta prefetching is to learn among all existing deltas and issue prefetch based on deltas with the highest confidence. As shown in the right figure,

with `+1, +2, +1, +2, ...` pattern, a delta prefetcher can learn that `+1` and `+2` has same confidence. It can then prefetch both `+1` and `+2`, increasing the coverage (though may decrease accuracy).

Similar to branch predictors, prefetchers can also learn deltas based on local contexts, such as access history of individual PCs, or history inside individual pages.

Timely prefetch

In Appendix A, we introduced the concept of late prefetches. An example is illustrated in figure below. Late prefetch happens because fetching data from lower levels of memory hierarchy requires time. For example, under the simplest pattern `+1, +1, +1, ...`, if the access interval is less than the fetch latency, next-line prefetcher can only provide zero coverage because all prefetches will be late. The principle of timely prefetch is to take prefetch latency into consideration. As shown in the right figure, timely prefetches can provide enough coverage.



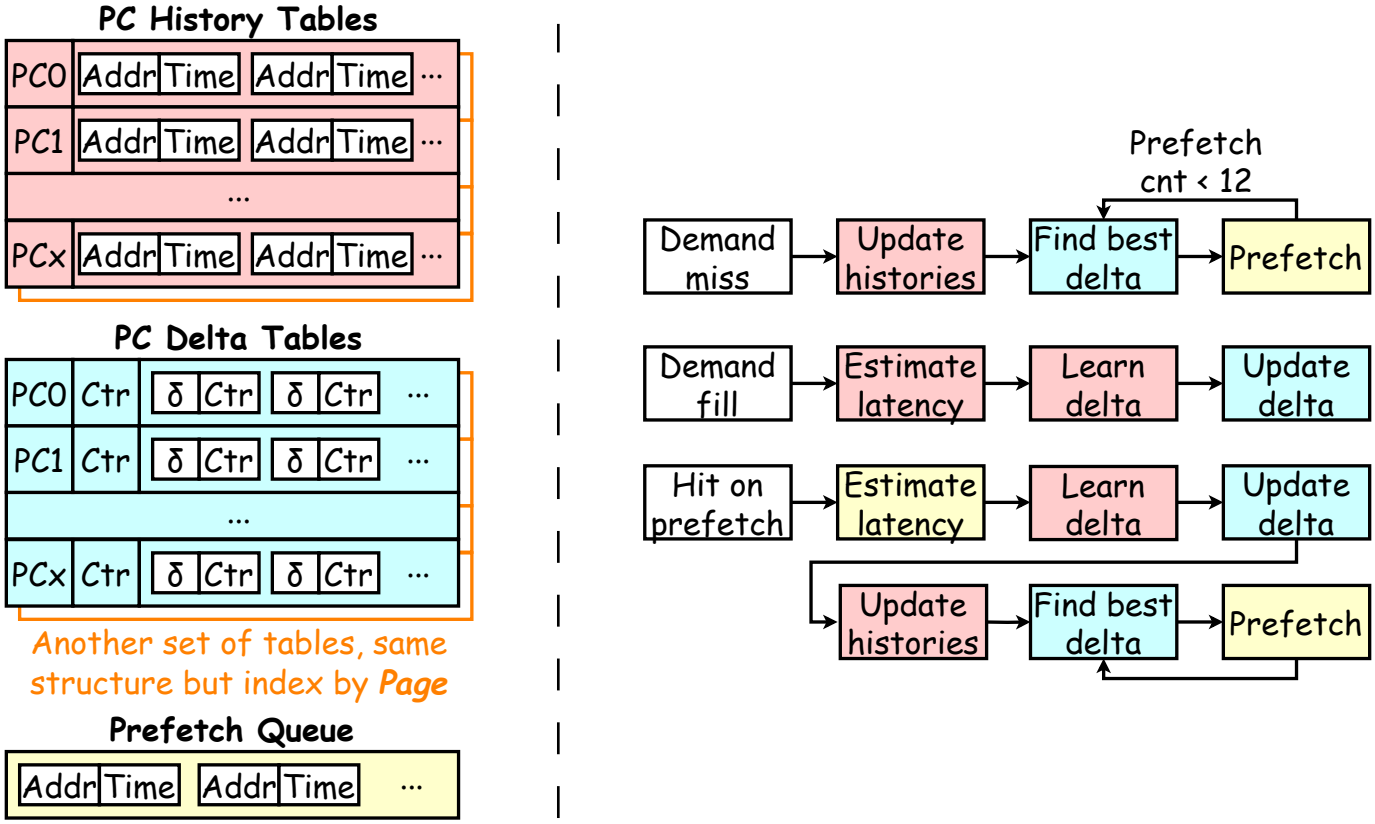
To find the best timely delta for prefetch, prefetchers can estimate the fetch latency from accesses and data fills. When a delta prefetcher records previous accesses, it will also record a **timestamp** of the access's arrival. When the data **is filled** into the cache (green part in the figure), the fetch latency can be estimated with `current timestamp - arrival timestamp`. Based on these information, when an access `B` arrives, the best delta `d` should meet the following constraints:

- In histories, the access interval between `A` and `A+d` is larger than the fetch latency (`+2` and `+3` in the figure).
- The `d` is the minimal delta that meet the above constraint (`+2 < +3`).

Once the best delta is learned, the prefetcher can issue timely prefetches.

Hyperion

Hyperion is a local-delta-based L1D prefetcher that generates timely prefetches to enhance both the accuracy and coverage. The figure below shows the overview of Hyperion's structure and methods.



Hyperion contains two kind of tables: history tables and delta tables. Each individual PC and page has its own history table and delta table. The history tables record the recent potential misses (demand miss and hit on prefetch) with its address and access timestamp. The delta tables record the recent occurred timely deltas. Each table has a total counter, and each delta has its local counter. These counters are used to calculate the confidence of deltas.

Three types of cache events can trigger Hyperion:

1. When there is a demand miss, Hyperion first updates the history tables (both PC and Page) with the address and timestamp. If a table is full, it will evict the earliest history (i.e., **FIFO eviction**). Then, Hyperion find both the PC delta table and Page delta table

for the delta with highest confidence, and prefetch with this delta. This find-prefetch cycle will perform 12 times.

2. When there is a demand fill (i.e., filling the data required by demand miss to cache), Hyperion will find the demand miss on this filled address in the history tables. The latency can be estimated by `lat = current - timestamp`. Then, it will scan the history tables and learn the best delta. Then, it will update the delta tables with this learnt delta. If the delta is already recorded in the table, the total counter and its local counter are plused by 1. If the delta is not recorded, the old delta with lowest confidence will be evicted, and the counters are updated (new local counter is 1).
3. When there is a hit on prefetched data, which indicates a potential miss, Hyperion will also estimate the latency (the timestamp of fetch-beginning can be found in the prefetch queue), learn best delta and update delta tables, like in scenario 2. It will also update history tables and issue prefetches, like in secnario 1.

> **Tip**
>
> - **Use block base address**: In cache context, all operations are in block granularity. The most important issue is that `notifyFill()` will pass block base address to the arguments. If you forget to use block base address, you may miss information in `notifyFill()`. So it's better to transform any address to block base address firstly (you can use `blockAddress()` or `blockIndex()`).
> - **Record more information**: The Hyperion is a hardware prefetcher. To mitigate its usage of hardware resources, it records only neccessary information. However, in your simulator, you can use much more memory. Feel free to record useful information to improve your prefetcher. The approaches Hyperion uses to mitigate hardware overheads are not neccessary in your implementation. We've listed them in next section.
> - **Use larger degree**: Hyperion is a degree-1 prefetcher, that is, for each selected delta, it prefetches only one block. However, increasing the degree may improve the performance under our specific test executables. You can use larger degree as long as it doesn't severely pollute the cache (leading to low accuracy and high cache miss rate). For example, `gem5` 's default stride perfetcher uses `degree = 4`.
> - **Use lower threshold**: Hyperion uses 0.8 as the L1D prefetch confidence threshold to avoid L1D cache pollution. However, such high confidence may

> lead to low coverage. You can use lower confidence during testing.

Features that are not required to implement

As a hardware design, Hyperion takes multiple approaches to reduce the hardware overheads. However, in `gem5` simulator, hardware overhead is not a critical concern if we primarily focus on the logical performance of a design. Moreover, some designs modify basic cache implementation, which can be challenging in `gem5`. For your convenience, you don't need to implement the following features:

- **Unified tables**: As described, Hyperion records histories and deltas for both individual PCs and individual pages. However, this can lead to redundant records. To reduce the used hardware, Hyperion merges the actual entries of PC and page tables into unified history/delta tables, and only uses separate indexing tables. In your implementation, you can just use two separate tables.
- **Prefetch buffer**: Hyperion issues multiple prefetches each round. However, hardware prefetch queue can be full and some prefetches may not have the chance to be issued. Hyperion introduces a prefetch buffer to temporarily records these prefetches. Our software impelementation can ignore this constraint.
- **Timestamp and latency recording**: Hyperion add some bits to cache blocks and MSHR entries to record the timestamps and latencies. This is more hardware-efficient than use new tables. However, it will be difficult to modify the cache and MSHR implementation in `gem5`. You can just use tables in your prefetcher to record timestamps and latencies.
- **Dynamic cache-level selection**: Hyperion will select the level of prefetches according to the delta confidence. Delta with low confidence may be filled into L2 cache. However, implementing this feature may be difficult in `gem5`, since the framework doesn't provide to prefetchers for a global access of all caches. You are not required to implement this feature.
- **Event on fill of prefetched data**: Hyperion will trigger additional events when a prefetched block is filled, due to limited PQ size. In your implementation, you don't need to implement this feature. In `notifyFill()`, you can use the interface `acc.pkt->cmd.isHWPrefetch()` to identify if the fill is from prefetching.